

Formation FPGA

TP2 : Compteurs

1. Introduction

L'objectif de ce second TP est la modélisation et la réalisation d'un compteur de temporisation en langage VHDL sous l'environnement Vivado. La manipulation consiste en la description matérielle du circuit, suivie de la validation de sa logique par simulation fonctionnelle (Testbench) et de l'analyse de ses contraintes temporelles. Enfin, l'implémentation et la programmation du Bitstream sur la carte FPGA Cora Z7 permettront la démonstration physique du système par le contrôle de l'allumage d'une LED via les boutons poussoirs.

2. Questions

2.1. Question 1

Afin d'obtenir la période d'une horloge matérielle fonctionnant à 100 MHz puis d'en déduire le nombre de cycles nécessaires pour une durée de 2 secondes, le calcul suivant est réalisé :

$$T = \frac{1}{f} = \frac{1}{10^8} = 10^{-8} s = 10 \text{ ns}$$

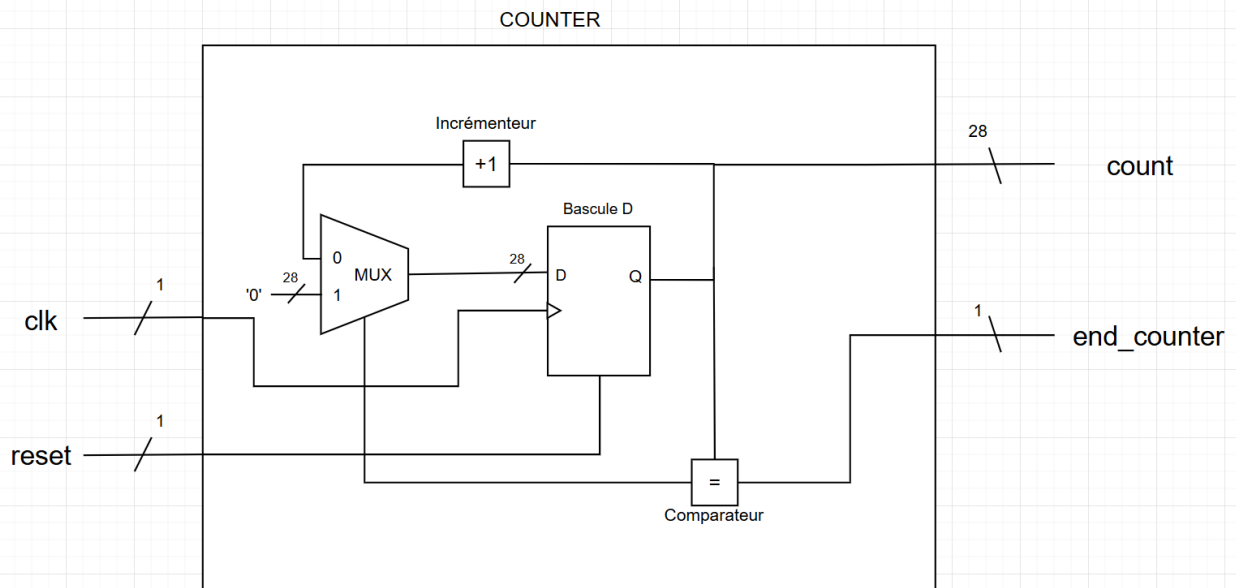
Il faut donc 200 millions de périodes afin d'obtenir les 2 secondes de temporisation.

Pour obtenir le nombre de bits nécessaires pour notre compteur, on aura besoin de la formule suivante :

$$N = \lceil \log_2(2 \cdot 10^8) \rceil$$
$$\log_2(200000000) = \frac{\ln(200000000)}{\ln(2)} \approx 28$$

Le compteur sera donc implémenté sur **28 bits**.

2.2. Question 2



Le schéma RTL ci-dessus est une première représentation théorique de notre code qui décrira le compteur en langage VHDL. Il est structuré autour de plusieurs blocs logiques :

- Le registre ou Bascule D mémorise la valeur actuelle du compteur à chaque front montant de l'horloge (clk). Son entrée de réinitialisation (reset) agit de manière asynchrone pour remettre instantanément le compteur à zéro.
- La boucle d'incrémentation (Bloc +1) récupère la valeur actuelle en sortie Q de la bascule, lui ajoute 1, et injecte ce résultat sur l'entrée 0 du Multiplexeur (MUX).
- Le bouclage autonome (Bloc =) ou le comparateur surveille la sortie Q.
- Tant que le seuil maximal n'est pas atteint, le comparateur sort un 0. Le MUX sélectionne son entrée 0 (count + 1), permettant au compteur de progresser.
- Dès que la valeur maximale du compteur est atteint, le comparateur sort un 1 (génération du signal end_counter). Ce 1 commande le MUX, qui bascule sur son entrée 1 (la constante '0'). Au coup d'horloge suivant, le compteur est réinitialisé à zéro pour recommencer un cycle.

2.3. Question 3

Le compteur sera remis à 0 de manière synchrone lorsque qu'il atteint la valeur de N-1 périodes, donc dans notre cas 199 999 999.

2.4. Question 4

Entrées	Sorties
clk (Horloge)	count (28 bits)
reset	end_counter (1 bit)

Le compteur sera en fonctionnement tant que l'horloge tournera et le reset non utilisé. La sortie « **count** » permettra de visualiser la valeur du compteur en simulation ou temps réel mais il n'est pas associé à une quelconque application externe pour le moment. Le signal « end_counter » pourra être utilisé tel un flag pour une autre application (comme le clignotement d'une LED).

2.5. Question 5

```
entity counter_unit is
    port (
        clk      : in std_logic;
        resetn    : in std_logic;
        end_counter : out std_logic;
        count     : out std_logic_vector(27 downto 0)
    );
end counter_unit;

architecture behavioral of counter_unit is

    --Declaration des signaux internes
    constant MAX : positive := 199999999;
    --constant MAX : positive := 10;    -- Pour test bench
    signal cpt_reg : unsigned (27 downto 0) := (others => '0') ;

begin

    --Partie sequentielle
    process(clk,resetn)
    begin
        if(resetn = '0') then
            cpt_reg <= (others => '0');
        elsif(rising_edge(clk)) then
            if(cpt_reg = MAX) then
                cpt_reg <= (others => '0');
            else
                cpt_reg <= cpt_reg + 1;
            end if;
        end if;
    end process;

    --Partie combinatoire
    end_counter <= '1' when (cpt_reg = MAX)
        else '0';

    count <= std_logic_vector(cpt_reg);
end behavioral;
```

L'entité « **counter_unit** » possède donc les entrées et sorties présentées précédemment.

On définit le comportement de l'entité en déclarant deux signaux internes comme la constante « **MAX** » pour la valeur maximale de compteur ainsi qu'un décimal de type unsigned pour mémoriser la valeur du compteur à chaque coup d'horloge.

Le type unsigned est ici plus pertinent qu'un type std logic vector en interne pour effectuer l'incréméntation en décimal plus facilement. Néanmoins dans le cas d'applications externes, la sortie est convertie en vecteur de bits pour une meilleure optimisation.

On utilise la logique séquentielle avec le process ayant sa liste de sensibilité pour effectuer le comptage et remettre à zéro selon les conditions comme le reset asynchrone ou la valeur MAX atteinte.

D'un autre côté, la logique combinatoire permet d'actionner ou non la sortie « **end_counter** » indépendamment du process synchrone pour une meilleure réactivité au niveau du fonctionnement du circuit.

2.6. Question 6

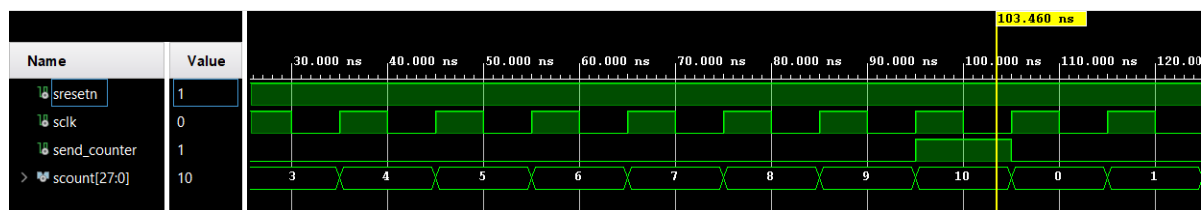
Le testbench à ce stade n'a pas réellement besoin de tests en dehors du process d'horloge, qui va automatiquement permettre au compteur de fonctionner. La valeur de sortie sera donc visualisable sur les chronogrammes.

Des tests automatiques type « assert » pourraient cependant être implémentées comme :

assert (unsigned(count) = MAX and end_counter = '1')

Cette instruction permet de s'assurer que le signal « end_counter » passe à 1 uniquement lorsque le compteur atteint sa valeur maximale par exemple.

2.7. Question 7



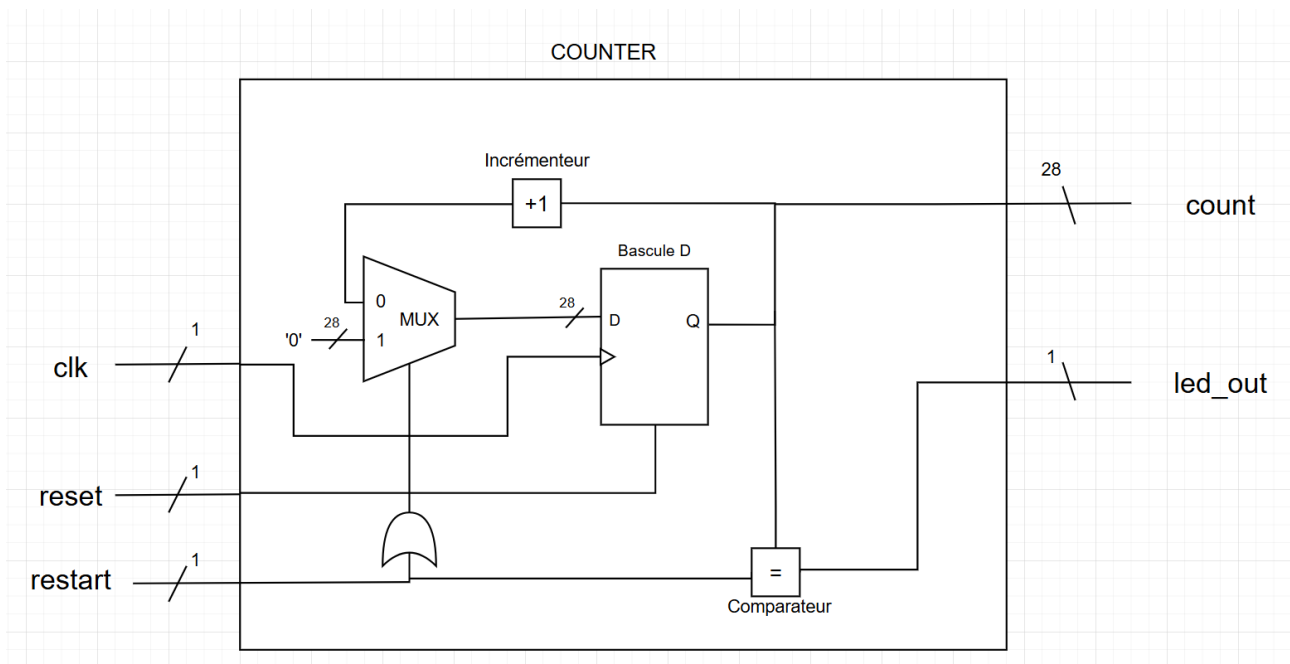
La constante MAX est définie ici à 10 pour avoir une simulation plus exploitable. Le signal « end_counter » passe à 1 quand le MAX est atteint, puis remise à zéro au prochain front d'horloge. La simulation est donc validée.

2.8. Question 8

```
#set_property -dict { PACKAGE_PIN G14 IOSTANDARD LVCMOS33 } [get_ports { led1_b }]; #IO_0_35 Sch=led1_b
set_property -dict {PACKAGE_PIN L14 IOSTANDARD LVCMOS33} [get_ports led_out]
```

On utilise le pin L14 pour allumer la LED1 directement reliée à la nouvelle sortie de l'entité « **counter_unit** » du même nom (obligatoirement!).

2.9. Question 9



Le schéma RTL reste relativement similaire au schéma précédent. Les seules différences sont situées au niveau des entrées et sorties. La sortie « end_counter », qui devient un signal interne dans cette configuration, est remplacée par la sortie « led_out » pour piloter une broche physique afin d’avoir une action visuelle. L’entrée « restart » est associée à un bouton poussoir permettant une interaction avec la logique du compteur à tout moment. C’est pourquoi le signal de commande du multiplexeur est précédé d’une porte **OR** afin de vérifier si un *restart* ou la valeur max du compteur est atteinte afin de remettre le compteur à zéro.

2.10. Question 10

```
entity counter_unit is
    port (
        clk          : in std_logic;
        resetn       : in std_logic;
        restart      : in std_logic;
        led_out      : out std_logic;
        count        : out std_logic_vector(27 downto 0)
    );
end counter_unit;

architecture behavioral of counter_unit is

    --Declaration des signaux internes
    constant MAX : positive := 199999999;
    --constant MAX : positive := 5;    -- Pour test bench
    signal cpt_reg : unsigned(27 downto 0) := (others => '0'); -- Bascule pour mémoriser la valeur précédente
    signal led_reg : std_logic := '0'; -- Bascule pour mémoriser la valeur précédente de la LED
    signal end_counter : std_logic := '0'; -- Flag quand valeur MAX atteinte
```

Les entrées/sorties sont également modifiées dans le code VHDL. Une autre bascule ou registre est utilisé afin de mémoriser la valeur précédente de la LED dans le cas d'une application de clignotement.

```
--Partie sequentielle
counter : process(clk, resetn)
begin
    if(resetn = '0') then
        cpt_reg <= (others => '0');
    elsif(rising_edge(clk)) then
        if(restart = '1') then
            cpt_reg <= (others => '0');
        elsif(cpt_reg = MAX) then
            cpt_reg <= (others => '0');
        else
            cpt_reg <= cpt_reg + 1;
        end if;
    end if;
end process;

--Logique combinatoire
end_counter <= '1' when (cpt_reg = MAX)
else '0';

LED_toggle : process(clk, resetn)
begin
    if(resetn = '0') then
        led_reg <= '0';
    elsif(rising_edge(clk)) then
        if(restart = '1') then
            led_reg <= '0'; -- Si restart, .
        elsif(end_counter = '1') then
            led_reg <= not led_reg; --On in
        end if;
    end if;
end process;

--Partie combinatoire
led_out <= led_reg;
count <= std_logic_vector(cpt_reg);
```

Afin d'optimiser la description matérielle, deux process distincts sont utilisés pour assurer une meilleure optimisation et lisibilité de code. Le premier est pratiquement identique à celui implémenté précédemment avec comme seul ajout la condition du signal « restart ».

Le deuxième process concerne le clignotement de la LED mais avec des conditions similaires avec le premier process afin d'assurer le fonctionnement synchrone avec l'horloge et asynchrone avec les boutons poussoirs.

Les sorties sont associées aux signaux internes dans la logique combinatoire.

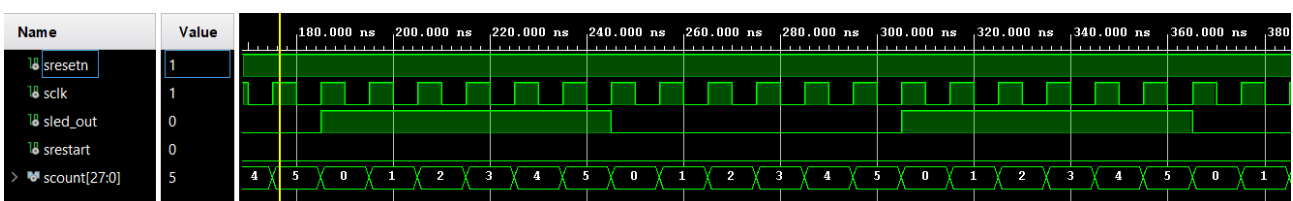
2.11. Question 11

```
set_property -dict {PACKAGE_PIN D19 IOSTANDARD LVCMOS33} [get_ports restart]
```

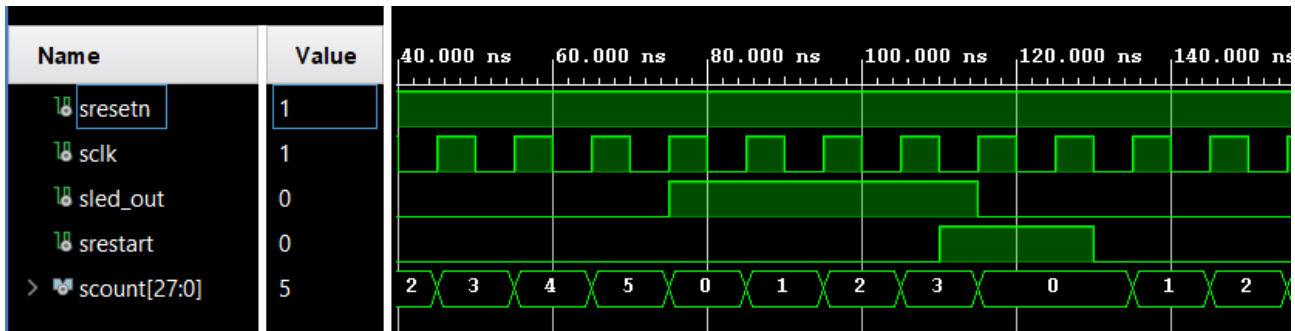
Le pin D19 est utilisé pour lier l'entrée « restart » au bouton poussoir BTN1.

2.12. Question 12

L'objectif est de tester les différentes entrées et sorties, dont « restart », « led_out » et « count ». Pour la simulation, la constante MAX sera égale à 5.



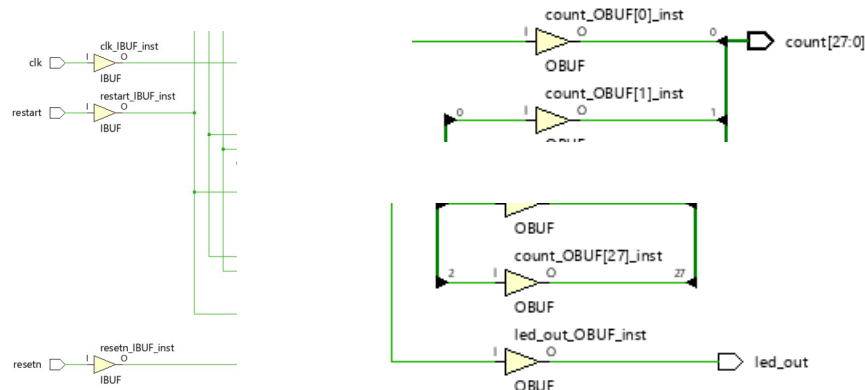
Sur ce premier chronogramme, la LED clignote bien pendant 2 secondes pour les états ON et OFF. Le compteur est mis à 0 à chaque fois que la valeur MAX est atteinte.



Le signal « restart » à l'état haut actionne bien la remise à zéro du compteur au prochain front montant d'horloge. Les simulations sont donc validées. (voir code test_bench fichier .vhd)

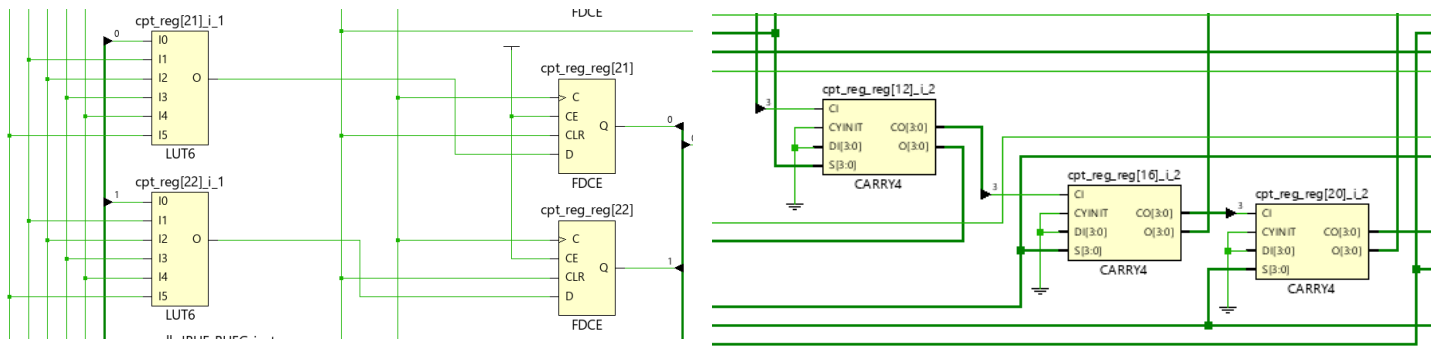
2.13. Question 13

Suite au lancement de la synthèse, le schéma technologique est obtenu avec de réels composants physiques :



Les buffers IBUF et OBUF font l'interface physique entre les broches externes de la carte Cora Z7 et la logique interne.

- Les IBUF reçoivent les signaux externes (clk, reset, restart), adaptent leur tension électrique, et les injectent à l'intérieur du FPGA.
- Les OBUF reçoivent les signaux logiques générés à l'intérieur du FPGA et fournissent la puissance électrique nécessaire pour piloter les composants externes (comme allumer la LED).



- Les LUT sont des mini-mémoires configurées pour agir comme des portes logiques. Dans notre cas, elles gèrent la décision du MUX et de la porte OR vérifiant si restart = 1 ou si MAX est atteint pour décider d'envoyer un 0 ou le résultat de l'additionneur.
- CARRY ou la chaîne de retenue est un composant matériel optimisé pour les opérations mathématiques rapides. Il réalise l'incrémentation (+1) sur les 28 bits du compteur en propageant la retenue aux autres blocs logiques.
- FDCE est la structure physique exacte de la Bascule D. C'est ici que la valeur actuelle du compteur est stockée (il y a 28 FDCE en parallèle pour le bus).

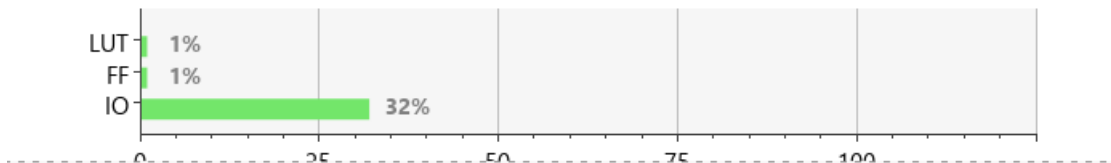
Le circuit forme donc une boucle fermée à travers ces composants :

1. Les signaux de contrôle extérieurs entrent par les IBUF (clk, reset, restart).
2. Les FDCE (Basculs) maintiennent la valeur actuelle pour chaque bit du compteur et l'envoient sur leur sortie Q.
3. Cette valeur Q entre simultanément dans les LUT (pour tester si on est égal à MAX) et dans les blocs CARRY (pour calculer la valeur +1).
4. Le résultat de cette décision combinatoire (LUT + CARRY) calcule la valeur +1 du compteur qui est réinjectée à l'entrée D des FDCE pour le prochain coup d'horloge.

2.14. Question 14

Summary

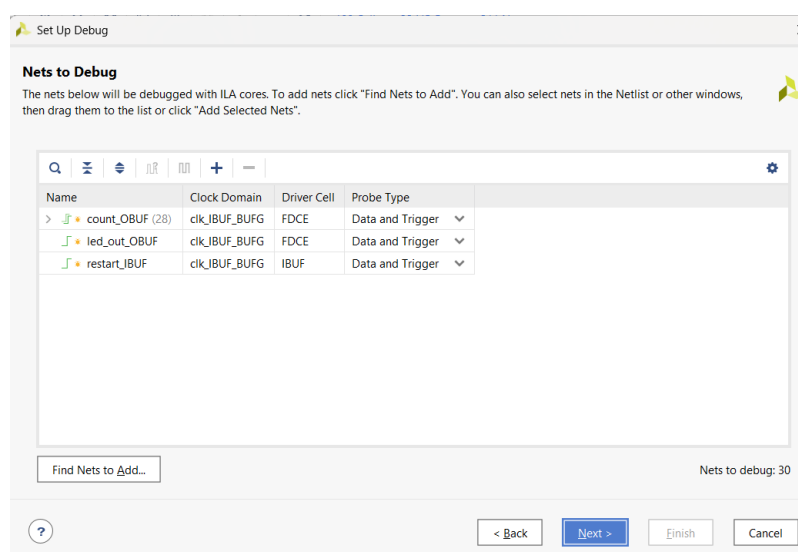
Resource	Utilization	Available	Utilization %
LUT	37	17600	0.21
FF	29	35200	0.08
IO	32	100	32.00

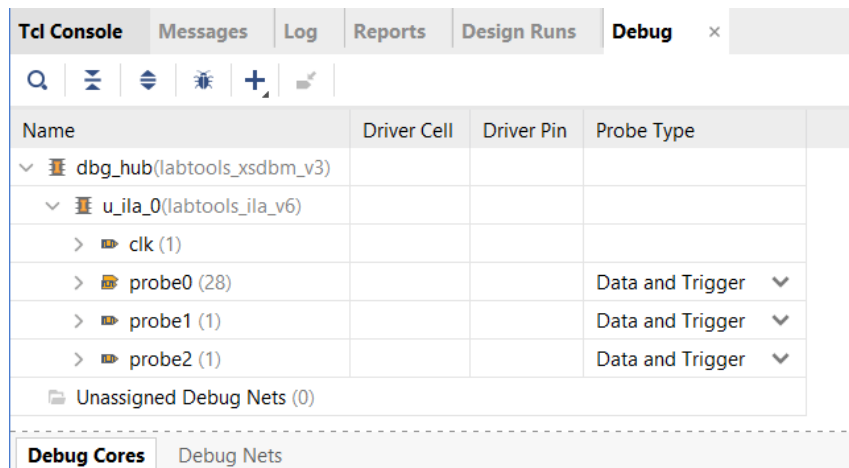


Les ressources utilisées sont visibles dans le rapport de synthèse et valident rigoureusement les prévisions de l'architecture théorique.

- Les LUT gèrent toute la logique combinatoire, notamment les opérations d'incrémenteur, comparateur de valeur ou bien l'aiguillage du multiplexeur. Pour un bus de 28 bits, 37 LUT utilisées paraît cohérent.
- 29 FF utilisées pour notre compteur codé sur 28 bits et pour atteindre la valeur maximale sans déborder.
- Les IO pour la totalité des broches externes physiques connectées sur le FPGA. Pour le bus de sortie sur 28 bits, la sortie LED, les entrées clk, restart, resetn, le résultat est conforme.

2.15. Question 15





La section « **Set Up Debug** » permet de configurer un ILA (*Integrated Logic Analyzer*). C'est un oscilloscope virtuel intégré directement à l'intérieur du FPGA pour visualiser les signaux physiques en temps réel pendant que le circuit tourne sur la carte Cora Z7. Les signaux clés de l'architecture seront donc surveillés après l'envoi du fichier bitstream sur la carte.

Dans la seconde image, on peut constater que Vivado génère directement un bloc de debug appelé « **u_ila_0** » afin de brancher les signaux sous forme de sondes matérielles appelées « **probes** » cadencées par l'horloge système « **clk** »

2.16. Question 16

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 2,088 ns	Worst Hold Slack (WHS): 0,027 ns	Worst Pulse Width Slack (WPWS): 2,750 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 3899	Total Number of Endpoints: 3883	Total Number of Endpoints: 2217

All user specified timing constraints are met.

Dans le résumé du rapport de timing (*Timing Summary*), il faut regarder deux indicateurs clés :

- WNS (Worst Negative Slack) pour le Setup : Il doit être positif. Cela signifie que les signaux combinatoires arrivent à destination avant le prochain front d'horloge. Il n'y a aucune violation de setup.
- WHS (Worst Hold Slack) pour le Hold : Il doit également être positif. Cela prouve que le signal reste stable assez longtemps après le front d'horloge pour être correctement mémorisé. Il n'y a aucune violation de hold.

Clock Summary (2)		Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock
Check Timing (31)		Path 21	2.088	4	5	58	<hidden>	<hidden>	5.627	1.188	4.439	8.0	sys_clk_pin	sys_clk_pin
Intra-Clock Paths		Path 22	2.088	4	5	58	<hidden>	<hidden>	5.627	1.188	4.439	8.0	sys_clk_pin	sys_clk_pin
dbg_hub/inst/BSCANID.u_xsdbm_i		Path 23	2.088	4	5	58	<hidden>	<hidden>	5.627	1.188	4.439	8.0	sys_clk_pin	sys_clk_pin
sys_clk_pin		Path 24	2.088	4	5	58	<hidden>	<hidden>	5.627	1.188	4.439	8.0	sys_clk_pin	sys_clk_pin
Setup 2,088 ns (10)		Path 25	2.098	4	5	58	<hidden>	<hidden>	5.616	1.188	4.428	8.0	sys_clk_pin	sys_clk_pin
Hold 0,027 ns (10)		Path 26	2.098	4	5	58	<hidden>	<hidden>	5.616	1.188	4.428	8.0	sys_clk_pin	sys_clk_pin
Pulse Width 2,750 ns (30)		Path 27	2.098	4	5	58	<hidden>	<hidden>	5.616	1.188	4.428	8.0	sys_clk_pin	sys_clk_pin

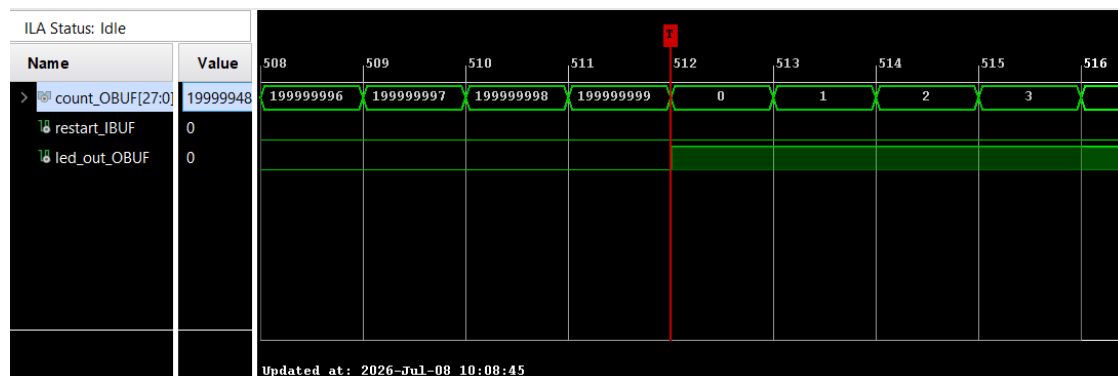
Le chemin critique est le chemin physique le plus lent de tout le circuit, celui qui limite la fréquence maximale de fonctionnement.

Dans notre architecture, le chemin critique part de la sortie Q d'une des bascules du compteur (FDCE), traverse toute la chaîne de retenue de l'additionneur (CARRY) ainsi que les LUT du comparateur (pour tester la condition = MAX ou la porte OR du restart), et se termine sur l'entrée D de la bascule du bit de poids fort du compteur.

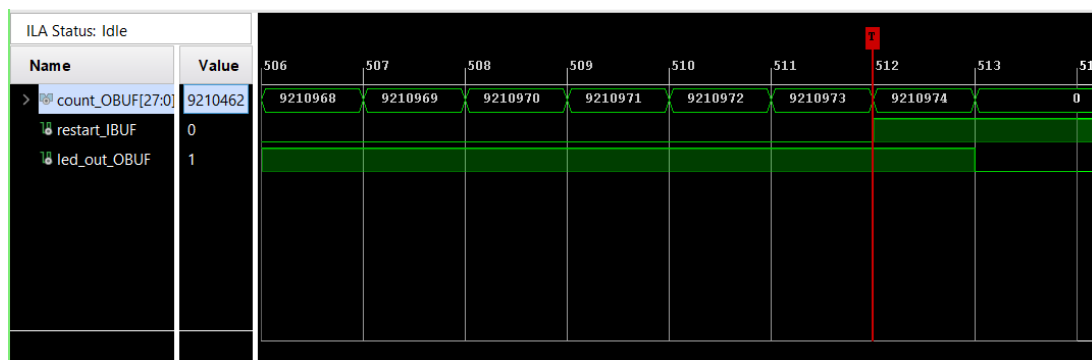
Dans le tableau ci-dessus, on se réfère au **Total Delay** qui est égal à **5,627 ns donc notre valeur de chemin critique**. Le slack étant positif, le circuit a une marge de sécurité et pourrait fonctionner à une fréquence d'horloge plus élevée.

2.17. Question 17

Après avoir envoyé le fichier Bitstream dans le FPGA, les différentes sondes placées sur le circuit vont envoyer les signaux obtenus en temps réel suite à différents triggers.



Une fois le seuil maximal atteint, l'état précédent de la LED est modifié au front montant d'horloge suivant.



Un trigger est ensuite placé quand le signal « restart » passe à 1 de manière asynchrone, on constate que le compteur ainsi que la LED repassent à 0 comme décrit dans le code VHDL.

Trigger Setup - hw_ila_1 × Settings - hw_ila_1 Status - hw_ila_1

🔍 + - 🔍

Name	Operator	Radix	Value	Port	Comparator Usage
led_out_OBUF	== (equal)	[B]	1	probe1[0]	1 of 1
restart_IBUF	==	[B]	1	probe2[0]	1 of 1

Le placement de trigger permet ainsi de capturer au moment souhaité les différents changements d'états du circuit.